JustCmul8 Viva: Basic Questions

One question per topic, simple answers

1. Project Overview

Q1. What is your project?

JustCmul8 is a free website to build and run simulations without coding. You drag boxes, join them, fill values and press Run.

Q2. What is Discrete Event Simulation?

Copying a real system on a computer. The clock jumps from one event to the next, like a customer arriving or a job finishing.

Q3. What problem does it solve?

Paid tools are costly, free tools need coding, and most need installing. Ours is free, no code and runs in the browser.

Q4. Who can use it and for what?

Students, teachers and small businesses. For example, a bank can test if one more counter will reduce the waiting line before spending money.

Q5. How does it work in short?

The diagram is turned into a Python SimPy program. It runs in the browser and the results show on a dashboard.

2. Diagrams

Q6. Context Diagram?

DFD level 0. Shows the whole system as one process with outside entities: Simulation Builder, Shared Link Viewer, Supabase Auth, and AI and Engine services.

Q7. DFD Level 1?

Breaks the system into 7 processes (P1 to P7) like Account, Project, Graph Design, Simulation, AI, Share and Results, with their data stores.

Q8. ER Diagram?

Shows tables and their links in Chen notation. Example: one User has one Profile and owns many Projects.

Q9. Class Diagram?

Shows classes with attributes and links, like User, Project, SimNode, SimEdge and SimulationRun.

Q10. State Transition Diagram?

Shows the states of a node during a run: IDLE, QUEUED, PROCESSING, COMPLETED, BLOCKED and FAILED.

Q11. Use Case Diagram?

Shows actors and what they can do. Simulation Builder builds and runs models. Shared Link Viewer can only view.

Q12. Activity Diagram?

Shows the user flow step by step: login, build graph, run, see results, AI help, export. Swimlanes show who does each step.

Q13. Component Diagram?

Shows the software parts and how they depend on each other, like editor, dashboard, state store and simulation worker.

Q14. Collaboration Diagram?

Shows objects and the numbered messages they pass during one simulation run.

Q15. Deployment Diagram?

Shows where it runs: user's browser, Vercel server and Supabase cloud.

Q16. Architecture Diagram?

Layered design: screens, state, services, execution in a Web Worker, and database. The special part is that simulation runs on the user's computer.

Q17. Gantt Chart?

Shows each task as a bar on a calendar from July 2026 to March 2027, with colours for each phase and diamonds for milestones.

Q18. Activity Network Diagram?

Shows the order of tasks, which run in parallel and where decisions are made during development.

3. Technology Stack

Q19. Next.js?

Framework for our pages, routing and API routes.

Q20. React?

Library to build the user interface with components.

Q21. TypeScript?

JavaScript with types. It catches mistakes before running.

Q22. Tailwind CSS?

Used for styling the screens.

Q23. React Flow?

Gives the drag and drop canvas with nodes and edges.

Q24. SimPy?

Free Python library that does the actual discrete event simulation.

Q25. Pyodide?

Python that runs inside the browser.

Q26. WebAssembly?

Fast code format that lets Pyodide run at near native speed in the browser.

Q27. Web Worker?

Background thread. The simulation runs here so the screen does not freeze.

Q28. Zustand?

Stores the app state like the graph and results.

Q29. Supabase?

Cloud backend for login, database and real time collaboration.

Q30. PostgreSQL and Row Level Security?

Our database. Row Level Security makes sure each user sees only their own data.

Q31. Google Gemini?

The AI that turns plain English into a model and suggests fixes. A router switches to another model if one fails.

Q32. Recharts?

Draws the charts on the results dashboard.

Q33. Pixi.js?

Draws the Digital Twin animation fast using WebGL.

Q34. jsPDF?

Makes the PDF report inside the browser.

Q35. Vercel?

Hosts our website.

Q36. Git and GitHub?

Version control to save code history and work together.

Q37. Jira?

Project management tool for tasks, sprints and bug tracking.

4. Other Key Topics

Q38. Which SDLC model did you use?

Iterative and incremental. We tested the risky engine first, then built one module at a time.

Q39. How many modules?

12 modules, 6 by each member.

Q40. How did you test?

We checked the engine against maths formulas (28 of 28 passed) and wrote 50 functional test cases.

Q41. What is Monte Carlo?

Running the model many times and showing the average with a 95 percent confidence range.

Q42. What is Little's Law?

$L = \lambda W$. Items in system equals arrival rate times time spent. We use it to check results.

Q43. What is a bottleneck?

The node with the longest waiting time. It slows the whole system.

Q44. What is COCOMO?

A model to estimate effort and cost from lines of code. Our code is 22.19 KSLOC.

Q45. What is Function Point Analysis?

Estimates size by features delivered. We got 281 function points.